ROS 2: motion planning

Dawid Seredyński

Warsaw University of Technology

June 2026

MoveIt★ – the standard *ROS 2* framework for robot manipulation:

- motion planning
- kinematics
- collision checking
- trajectory generation
- execution
- for: robotic arms, grippers, mobile manipulators, and similar systems

MoveIt OSS (open source) is free under a BSD license for industrial, commercial, and research use

MoveIt Pro★

- paid annual license + paid services
- behaviors, skills, behavior trees, mobile manipualtion and navigation, ML, digital twin

MoveIt:

- nodes
- tools
- *plugin*-based (*pluginlib*)
- uses *OMPL*

The Open Motion Planning Library (OMPL) ★

- many state-of-the-art sampling-based motion planning algorithms
- no code related to, e.g., collision checking or visualization

Note about *MoveIt* documentation:

- docs are distributed along multiple sites (usually redundant)
- some diagrams and descriptions are not migrated from *ROS 1*
- *MoveIt* for *ROS 2* is called *MoveIt 2*

MoveIt pipeline★

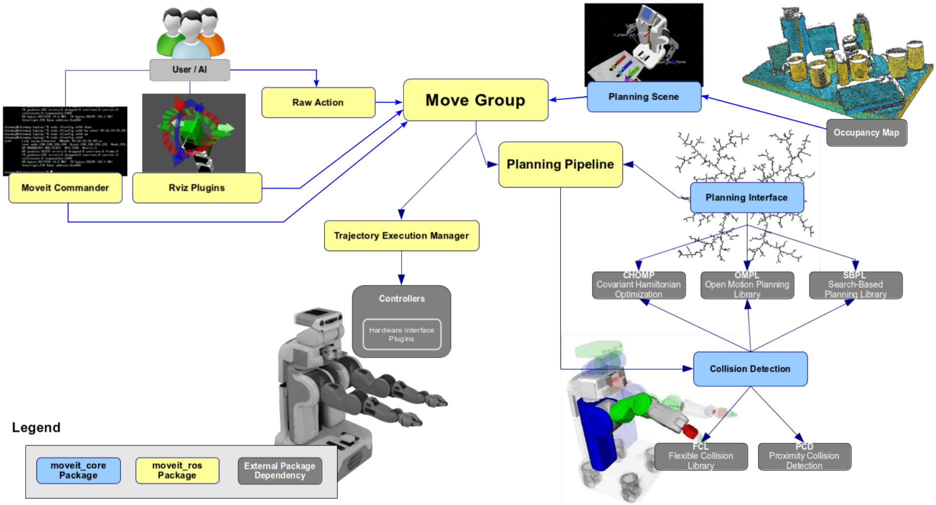


image source: https://moveit.picknik.ai/main/doc/concepts/concepts.html *move_group*★ is a central *node* of *MoveIt*

- provides *services* and *action*s callable from:
  - *node*s in C++ – through `move_group_interface`
  - *node*s in Python – through `MoveItPy`
  - *rviz2* – through *MotionPlanning* visualization panel
  - *CLI*
- reads sensors *topics* and manages the planning scene
- perform path / trajectory planning and IK
- commands trajectory execution to *ros2_control*

move_group may run several child *node*s, including:

- planning scene monitor
- a client *node* for interacting with *controller_manager*

Note: <u>do not</u> set the name of *move_group node*

- this will set the same name for all child nodes and cause problems!
- use *namespace* instead, if needed (multiple robots?)

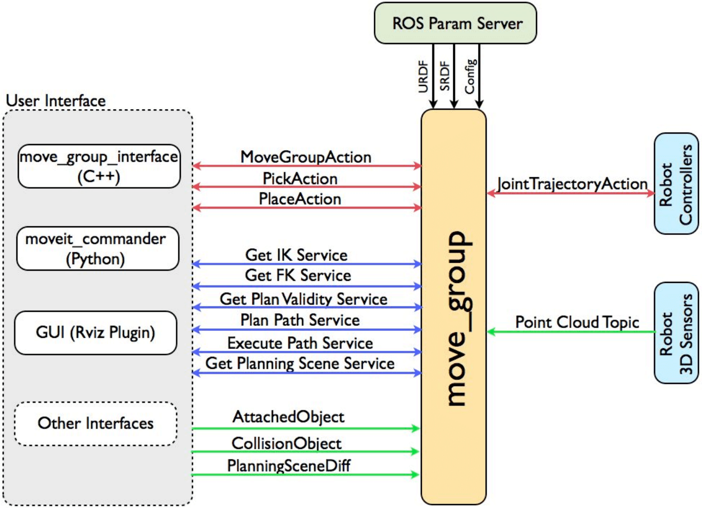


image source: `https://moveit.picknik.ai/main/doc/concepts/move_group.html`

Figure: The *move_group node* (note: this diagram is for ROS 1)

MoveIt Setup Assistant★

- used to create or edit a robot's MoveIt configuration package
- GUI tool
- input: robot description (*URDF* / *xacro*)
- generates the semantic (*SRDF*) and `.yaml` configuration files needed by:
  - ◇ *move_group*
  - ◇ *rviz2* Motion Planning panel
  - ◇ kinematics solvers, planners
  - ◇ trajectory execution
- output (*SRDF*, `.yaml`, *xacro*):
  - ◇ planning groups
  - ◇ end effectors
  - ◇ disabled collision pairs
  - ◇ named poses
  - ◇ passive joints
  - ◇ kinematics settings
  - ◇ controller mappings

MoveIt Setup Assistant generates a *package*

```
my_robot_moveit_config/
├── config/
│   ├── joint_limits.yaml
│   ├── kinematics.yaml
│   ├── moveit_controllers.yaml
│   ├── my_robot.srdf
│   ├── my_robot.urdf.xacro
│   ├── my_robot.ros2_control.xacro
│   ├── ompl_planning.yaml
│   ├── ros2_controllers.yaml
│   ├── sensors_3d.yaml
│   └── ...
├── launch/
│   ├── move_group.launch.py
│   ├── rviz.launch.py
│   ├── demo.launch.py
│   └── ...
├── .setup_assistant
├── package.xml
└── CMakeLists.txt
```

MoveIt and simulation

MoveIt Setup Assistant does not generate controllers for simulation

`move_group` launch file does not read *parameter* `use_sim_time`

This causes the following error:

```
[rviz2-10] [WARN] [1780838151.257620804] [rviz2.moveit.ros.planning_scene
_monitor]: Maybe failed to update robot state, time diff: 1780838135.251s
```

Fixes:

- rewrite the function `generate_move_group_launch`★
- or download the `move_group` launch file★

Running *MoveIt*

Minimal set of nodes for running *MoveIt*:

- *controller_manager* (*ros2_control* or gz_ros2_control)
  - *hardware components*
  - `joint_state_broadcaster`
  - trajectory controllers
- `robot_state_publisher`
- *move_group* and all its child *nodes*

Usual additions:

- *rviz2* with Motion Planning *plugin*
- perception nodes

Using *MoveIt* – API

In *rviz2* + *MotionPlanning* panel★:

- most of *MoveIt* functions available from GUI

In a C++ *node*★:

- through `MoveGroupInterface` class

In a Python *node*★:

- through `MoveItPy` class in `moveit.planning` module
- it is convenient to use `moveit_py.yaml` (the configuration for `moveit_py`)

By calling a service or action

- C++ / Python classes, *rviz2 plugin* are using *move_group* services and actions

Configuring *MoveIt* is difficult, but it is a one-time procedure

Planning Scene Monitor★

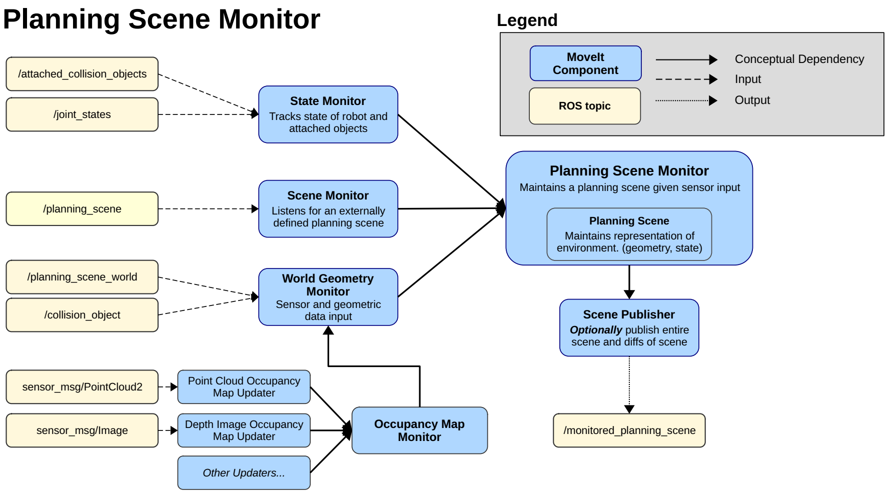
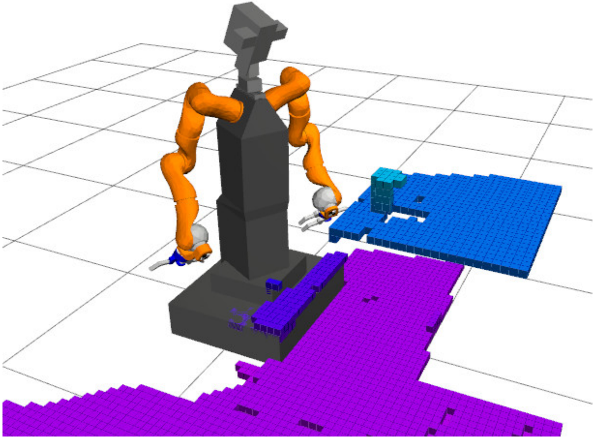


image source: `https://moveit.picknik.ai/main/doc/concepts/planning_scene_monitor.html`

3D sensor data → octomap

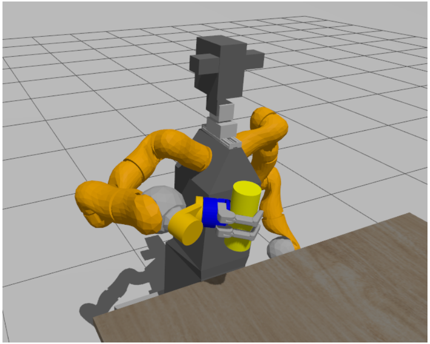


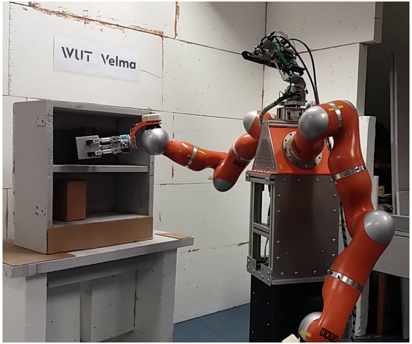
Sense, plan and act

Typical use:

1. sense
   - use sensors' data to build environment model
   - send the model to *move_group node*
2. plan
   - determine starting and goal configurations
   - inform the *move_group node* about attached objects and constraints (for the goal and trajectory)
   - trigger motion planning
3. act
   - execute the generated path

WUT Velma robot – simulation





https://github.com/dseredyn/wut_velma_ws

Questions?